Practical Session 2: Transformers, APIs, and RAG

Hands-On: Attention by Hand, Cost Estimation, and a Minimal RAG Pipeline

Juan F. Imbet

Paris Dauphine – PSL University

Large Language Models in Finance

Session Overview

Duration: 1 hour (50 min problems + 10 min debrief)

Three activities:

1. **Problem 1** (15 min): Trace scaled dot-product attention by hand on a 3-token financial sequence. Compute $QK^\top$, scale, softmax, multiply V.
2. **Problem 2** (15 min): Estimate API costs for a realistic compliance workload. Compare GPT-4o, Claude 3 Haiku, and self-hosted Llama 3.
3. **Case Study** (20 min): Run a minimal RAG pipeline in Python using `sentence_transformers`, `faiss`, and `anthropic`. Answer three diagnostic questions about retrieval behaviour.

Discussion (10 min)

When would you choose BERT vs. GPT vs. RAG for a financial NLP task? Give a concrete scenario for each.

Recap: Scaled Dot-Product Attention

You will need this formula for Problem 1.

Given: query matrix Q, key matrix K, value matrix V, head dimension d_k.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Steps:

1. Compute $S = QK^\top$ (raw score matrix, shape $n \times n$)
2. Scale: $\tilde{S} = S/\sqrt{d_k}$
3. Apply softmax row-wise to get attention weights A
4. Output: $O = AV$

Softmax reminder:

$$\text{softmax}(\mathbf{x})_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Numerical stability tip: subtract $\max(\mathbf{x})$ before exponentiating.

Recap: BPE Tokenisation and Cost Formula

You will need this for Problem 2.

Token conversion rule of thumb:

$$\text{tokens} \approx 1.3 \times \text{words}$$

API cost model:

$$\text{cost} = N_{\text{in}} \cdot c_{\text{in}} + N_{\text{out}} \cdot c_{\text{out}}$$

where N_{in}, N_{out} are token counts and c_{in}, c_{out} are per-token prices in USD per million tokens (MTok).

Model	**Input ($/MTok)**	**Output ($/MTok)**
GPT-4o	$5.00	$15.00
Claude 3 Haiku	$0.25	$1.25
Llama 3 (A100)	$0 (compute only)	$0

Self-hosted cost: an A100 GPU costs approximately $3/hr on cloud spot markets.

Problem 1: Attention by Hand — Setup

Sequence: 3 tokens from a financial sentence: `"revenue"`, `"increased"`, `"significantly"`.

Head dimension: $d_k = 2$.

Given matrices (small integers for tractability):

$$Q = \begin{pmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 1 \end{pmatrix}, \quad K = \begin{pmatrix} 1 & 1 \\ 0 & 2 \\ 1 & 0 \end{pmatrix}, \quad V = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}$$

Rows of Q, K, V correspond to tokens `revenue` (row 1), `increased` (row 2), `significantly` (row 3).

Interpretation

$d_k = 2$ is unrealistically small; in GPT-3 $d_k = 64$; in GPT-4 $d_k \approx 128$. The algebra is identical — we just keep arithmetic manageable.

Problem 1: Tasks

Complete the following steps. Show your working.

Task A. Compute $S = QK^\top$ (a 3×3 matrix).
Recall: entry $(i, j) = \mathbf{q}_i^\top \mathbf{k}_j$.

Task B. Scale by $\sqrt{d_k} = \sqrt{2} \approx 1.414$.
Report $\tilde{S} = S/\sqrt{2}$ to two decimal places.

Task C. Apply softmax row-wise to $\tilde{S}$ to obtain the attention weight matrix A (each row sums to 1). Round to three decimal places.

Task D. Compute the output $O = AV$.
Each row of O is the attention-weighted sum of value vectors.

Discussion questions:

- Which value vector does token `revenue` attend to most heavily?
- What would happen to the attention weights if $d_k = 64$ instead of 2, with the same raw dot products? Would they be sharper or flatter?

Problem 2: API Cost Estimation

Scenario: A compliance team at a large European bank needs to process $N = 5{,}000$ annual report filings (10-K equivalents). Each filing has approximately 80,000 words. The pipeline uses:

- A system prompt of 1,000 tokens (analytical instructions, fixed)
- Full filing text as input (converted to tokens)
- 600 output tokens per filing (structured compliance summary)

Task A. Compute the total cost of processing all 5,000 filings using **GPT-4o** ($5/MTok input, $15/MTok output).

Task B. Compute the same cost using **Claude 3 Haiku** ($0.25/MTok input, $1.25/MTok output).

Task C. The team considers self-hosting Llama 3 on an A100 GPU rented at $3/hour. The pipeline can process 200 filings per hour. What is the total compute cost? Under what conditions would self-hosting be preferred?

Bonus: If the system prompt is cacheable and Anthropic offers 90% discount on cached input tokens, recompute the Claude 3 Haiku cost.

Case Study: A Minimal RAG Pipeline for SEC Filings

Goal: build a working RAG pipeline that retrieves relevant passages from a small corpus of SEC filing excerpts and uses Claude to answer a question.

Stack:

- `sentence_transformers` — bi-encoder embeddings (SBERT)
- `faiss` — approximate nearest-neighbour index
- `anthropic` — generation with the Claude API

Corpus: 5 SEC filing passages (pre-loaded in the code).

What you will do:

1. Run the provided Python code
2. Observe which passage is retrieved for a given query
3. Answer three diagnostic questions about retrieval behaviour

Prerequisites

You need an `ANTHROPIC_API_KEY` environment variable set.
Install: `pip install sentence-transformers faiss-cpu anthropic`

Case Study: The Code

```
from sentence_transformers import SentenceTransformer
import faiss, numpy as np, anthropic

PASSAGES = [
    "Revenue for FY2023 was $4.2 billion, up 12% year-over-year.",
    "Operating expenses increased to $1.8 billion due to R&D
        investment.",
    "Net income declined 5% to $820 million, impacted by higher
        interest expense.",
    "The company holds $2.1 billion in cash and equivalents.",
    "Guidance for FY2024 projects revenue growth of 8-10%.",
]

model = SentenceTransformer("all-MiniLM-L6-v2")
embs = model.encode(PASSAGES, normalize_embeddings=True).astype("
    float32")
index = faiss.IndexFlatIP(embs.shape[1])
index.add(embs)

def rag_query(query: str, k: int = 2) -> str:
    q_emb = model.encode([query], normalize_embeddings=True).astype
        ("float32")
    _, ids = index.search(q_emb, k)
    context = "\n".join(f"[{i+1}] {PASSAGES[ids[0][i]]}" for i in
```

Case Study: Diagnostic Questions

Run the code for the query `"What was the net income?"` and then answer:

Q1. Which passage(s) were retrieved (i.e., which indices i did FAISS return)? Does the answer from Claude correctly cite those passage numbers? Why or why not?

Q2. Change $k = 2$ to $k = 1$. Does the answer change? Change it to $k = 5$ (all passages). How does including more context affect the quality and length of the answer?

Q3. Change `temperature=0.0` to `temperature=0.7`. Run the same query three times. Do you observe variation in the output? Is that variation desirable for this task? What does this tell you about the appropriate temperature for structured financial extraction?

Q4 (Hallucination). Modify the code to skip retrieval and send only the raw question to Claude with no context. Does the model still produce an answer?

- What type of hallucination is this (intrinsic or extrinsic)?

Discussion: When to Choose BERT vs. GPT vs. RAG

For each scenario below, identify which approach you would use and why.

1. A compliance officer needs to classify 50,000 news headlines per day as *material* or *non-material* for insider-trading monitoring purposes. Latency target: under 50 ms per headline. Training data: 10,000 labelled examples.
2. A portfolio manager wants a system that can answer questions about any 10-K filing in the portfolio in real time, using the most recent version of each filing. The filings were not part of any model's training data.
3. An investment bank needs to draft the "Risk Factors" section of a new bond prospectus given a structured data dump of the issuer's financial statements and a comparable set of five precedent transactions.

Frame your answer around:

Latency, available labelled data, generation vs. classification, need for up-to-date information, cost, and auditability.

Solution: Problem 1 — Attention by Hand

Task A: $S = QK^\top$

$$S = \begin{pmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 1 \\ 1 & 2 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 1 \\ 3 & 2 & 2 \\ 1 & 2 & 0 \end{pmatrix}$$

Task B: $\tilde{S} = S/\sqrt{2}$

$$\tilde{S} \approx \begin{pmatrix} 0.71 & 0.00 & 0.71 \\ 2.12 & 1.41 & 1.41 \\ 0.71 & 1.41 & 0.00 \end{pmatrix}$$

Task C: Softmax row-wise (row 1 example: $e^{0.71} + e^0 + e^{0.71} = 2.034 + 1 + 2.034$)

$$A \approx \begin{pmatrix} 0.400 & 0.200 & 0.400 \\ 0.561 & 0.220 & 0.220 \\ 0.260 & 0.529 & 0.211 \end{pmatrix}$$

Task D: $O = AV$

Solution: Problem 2 — API Cost Estimation

Token counts per filing:

- Input: $80{,}000 \times 1.3 = 104{,}000$ tokens from filing text
- $+1{,}000$ system prompt $= 105{,}000$ input tokens per filing
- Output: 600 tokens per filing

Task A — GPT-4o:

$$\text{cost} = 5{,}000 \times \left(\frac{105{,}000}{10^6} \times 5.00 + \frac{600}{10^6} \times 15.00 \right) = 5{,}000 \times (0.525 + 0.009) =$$

Task B — Claude 3 Haiku:

$$\text{cost} = 5{,}000 \times \left(\frac{105{,}000}{10^6} \times 0.25 + \frac{600}{10^6} \times 1.25 \right) = 5{,}000 \times (0.02625 + 0.00075)$$

Task C — Self-hosted Llama 3 (A100 at \$3/hr, 200 filings/hr):

$$\text{time} = 5{,}000/200 = 25 \text{ hr}, \quad \text{cost} = 25 \times 3 = \$75$$

Bonus — Prompt caching (90% off system prompt for Claude Haiku):

$$\left(\frac{104{,}000}{} \right. \qquad \frac{1{,}000}{} \qquad \left. \frac{600}{} \right)$$

Wrap-Up: Key Takeaways from This Session

From Problem 1:

- Attention is a weighted average of value vectors; the weights come from (softmaxed) scaled dot products between queries and keys
- Larger d_k sharpens the softmax — which is why we normalise by $\sqrt{d_k}$
- The mechanism is fully differentiable and can be traced algebraically

From Problem 2:

- Cost differences between API providers can be 20–35$\times$ for the same workload
- Self-hosting is competitive when the workload is large and latency is flexible
- Prompt caching for fixed system prompts provides significant savings

From the Case Study:

- RAG reliably retrieves semantically relevant passages, even from small corpora
- k and temperature are meaningful hyperparameters with measurable output effects